© 2026 TravelX Team

SOURCE CODE COPYRIGHT DOCUMENT

TravelX

Real-Time Ride Booking Application

Project Name	**TravelX – Real-Time Ride Booking Application**
Developers	Ujjwal Mutneja · Vaibhav Tiwari · Sahibjot Singh · Harshita Bansal
Tech Stack	MERN (MongoDB, Express.js, React.js, Node.js)
Document Type	Source Code Copyright Submission
Year	2026

KEY FEATURES

- Real-time ride booking via Socket.io
- Dynamic fare calculation engine
- Dual wallet — captain + commission ledger
- Stripe payment with server-side verify
- JWT auth + token blacklist on logout
- Admin analytics & fleet dashboard
- Geospatial captain matching (MongoDB)
- Cryptographic OTP ride verification
- Cashout & bank settlement system
- Live GPS tracking (TomTom SDK)
- Email notifications (Nodemailer)
- Responsive mobile-first React UI

TABLE OF CONTENTS

1. System Architecture
2. Core Modules
- 2.1 Ride Booking & Driver Matching
- 2.2 Wallet & Commission System
- 2.3 Payment Integration (Stripe)
- 2.4 Real-Time Communication (Socket.io)
- 2.5 GPS Tracking & Maps
- 2.6 Authentication (JWT + Bcrypt)
- 2.7 Admin Analytics
3. Core Algorithms
- 3.1 Geospatial Driver Search
- 3.2 Dynamic Fare Calculation
- 3.3 Cryptographic OTP Generation
- 3.4 Ride Lifecycle State Machine
- 3.5 JWT Authentication with Blacklist
- 3.6 Bcrypt Password Hashing Guard
4. Frontend Logic & UI Algorithms
- 4.1 Debounce-Controlled Location Search
- 4.2 Captain GPS Polling Engine
- 4.3 Live Map Route Rendering
- 4.4 Live Distance Matrix Fetcher
5. Legal Note

© 2026 TravelX Team

01 System Architecture

FRONTEND

React.js (Vite) with Context API for global state. GSAP handles animated panel transitions. TomTom Web SDK renders live maps. Stripe Elements provides secure card UI. Axios (REST) + Socket.io-client (WebSocket).

BACKEND

Express.js REST API with MVC structure — controllers, services, models, middlewares, routes. `express-validator` enforces input validation. Nodemailer sends transactional emails.

DATABASE

MongoDB via Mongoose. Five core collections: `users`, `captains`, `rides`, `blacklisttokens`, `companywallet`. Captain documents use GeoJSON-compatible location for native geospatial queries.

REAL-TIME LAYER

Socket.io server. Socket IDs are stored per-document in MongoDB. On disconnect, IDs are cleared and captain availability is set to `false`. Events: `new-ride`, `ride-confirmed`, `ride-started`, `ride-ended`, `payment-success`.

External APIs

API	Purpose
TomTom Geocoding API	Convert address text → lat/lng coordinates
TomTom Routing API	Calculate driving distance & time between two points
TomTom Search API	Autocomplete location suggestions
Stripe API	Payment intent creation and server-side verification
Nodemailer / SMTP	Transactional emails at ride start and ride end

© 2026 TravelX Team

02 Core Modules

2.1 Ride Booking & Driver Matching

When a user submits a ride request, both addresses are geocoded, a ride document is created with `status: 'pending'`, and MongoDB is queried for active captains within a 45 km sphere. Each matching captain receives a real-time dispatch via WebSocket. The first captain to confirm atomically locks the ride and broadcasts rejection to all others.

backend/controllers/ride.controller.js JAVASCRIPT

```javascript
module.exports.createRide = async (req, res) => {
    const { pickup, destination, vehicleType } = req.body;

    // Geocode both addresses to coordinates
    const pickupCoords = await mapServices.getAddressCoordinate(pickup);
    const destCoords   = await mapServices.getAddressCoordinate(destination);

    // Create ride document (status: 'pending')
    const ride = await rideService.createRide({
        user: req.user._id, pickup, destination, vehicleType,
        pickupLat: pickupCoords.lat,  pickupLng: pickupCoords.lng,
        destLat:   destCoords.lat,    destLng:   destCoords.lng,
    });

    // Geospatial query — find captains within 45 km radius
    const captainsInRadius = await mapServices.getCaptainInTheRadius(
        pickupCoords.lng, pickupCoords.lat, 45
    );

    // Broadcast new-ride event to every matched captain socket
    captainsInRadius.map(captain =>
        sendMessageToSocketId(captain.socketId,
            { event: 'new-ride', data: rideWithUser })
    );
    res.status(201).json(ride);
};

module.exports.confirmRide = async (req, res) => {
    const { rideId } = req.body;
    // First captain to confirm wins — DB-level race condition guard
    const ride = await rideService.confirmRide({ rideId, captain: req.captain });
    sendMessageToSocketId(ride.user.socketId, { event: 'ride-confirmed', data: ride });
    res.status(200).json(ride);
};
```

2.2 Wallet & Commission System

On ride completion, the platform deducts the configurable commission percentage. Logic branches by payment mode: for card/online payments the platform holds the full fare and credits the captain's net share; for cash the captain already holds the fare, so only the commission debt is deducted. The company wallet ledger is incremented in parallel.

backend/services/ride.service.js — JAVASCRIPT

```javascript
const commissionRate = parseFloat(process.env.COMMISSION_RATE) || 0.20;
const commission     = ride.fare * commissionRate;
const captainShare   = ride.fare - commission;

if (ride.paymentMode === 'card' || ride.paymentMode === 'online') {
    // Platform holds fare — credit captain's net share to wallet
    await captainModel.findByIdAndUpdate(captain._id, {
        $inc: { wallet: captainShare, totalEarnings: captainShare }
    });
} else {
    // Captain collected cash — deduct only the commission debt
    await captainModel.findByIdAndUpdate(captain._id, {
        $inc: { wallet: -commission, totalEarnings: captainShare }
    });
}
// Increment central company ledger simultaneously
await CompanyWallet.findByIdAndUpdate(companyWallet._id, {
    $inc: { totalBalance: commission, totalCommissionEarned: commission, totalRidesHandled: 1 }
});
```

backend/controllers/captain.controller.js — Cashout — JAVASCRIPT

```javascript
module.exports.handleCashout = async (req, res) => {
    const threshold = parseFloat(process.env.CASHOUT_THRESHOLD) || 100;
    if (captain.wallet < threshold)
        return res.status(400).json({ message: `Minimum ₹${threshold} required.` });

    // Mask account number — show only last 4 digits in audit trail
    const masked = accNo.length >= 4
        ? '*'.repeat(accNo.length - 4) + accNo.slice(-4) : accNo;

    await SettlementHistory.create({ type: 'CASHOUT', status: 'SUCCESS', masked... });
    captain.wallet = 0;   // Zero-out wallet on ledger commitment
    await captain.save();
    res.status(200).json({ message: 'Cashout successful', wallet: 0 });
};
```

2.3 Payment Integration (Stripe)

After the rider pays via Stripe Elements, the backend independently retrieves the Payment Intent from Stripe's API to verify `status === 'succeeded'`. This server-side check prevents payment bypass attacks. On success, the ride document is updated and the captain is notified via WebSocket.

backend/controllers/paymentController.js JAVASCRIPT

```javascript
const confirmPayment = async (req, res) => {
    const { rideId, amount, paymentIntentId } = req.body;
    const ride = await rideModel.findById(rideId).populate('captain');

    // Server-side Stripe verification — never trust client status claim
    const paymentIntent = await stripe.paymentIntents.retrieve(paymentIntentId);
    if (paymentIntent.status !== 'succeeded')
        return res.status(400).json({ message: 'Payment not verified' });

    await rideModel.findByIdAndUpdate(rideId, {
        paymentMode: 'card',  paymentStatus: 'paid',
        orderId: crypto.randomUUID(),  paymentID: paymentIntent.id,
    });
    sendMessageToSocketId(ride.captain.socketId, { event: 'payment-success', data: { amount } });
    res.json({ success: true });
};
```

2.4 Real-Time Communication (Socket.io)

The Socket.io server maintains a persistent bidirectional channel with every client. On `join`, the socket ID is written to the user/captain MongoDB document. On `update-location-captain`, coordinates are updated in real time. On disconnect, the socket ID is cleared and the captain is marked unavailable.

backend/socket.js JAVASCRIPT

```javascript
io.on('connection', (socket) => {

    // Bind socket ID to identity in DB on session start
    socket.on('join', async ({ userId, userType }) => {
        if (userType === 'user')
            await userModel.findByIdAndUpdate(userId, { socketId: socket.id });
        else if (userType === 'captain')
            await captainModel.findByIdAndUpdate(userId, { socketId: socket.id });
    });

    // Continuous fleet GPS update — persists live position
    socket.on('update-location-captain', async ({ userId, location }) => {
        await captainModel.findByIdAndUpdate(userId,
            { $set: { location: { lat: location.lat, lng: location.lng } } },
            { new: true, runValidators: true }
        );
    });

    // Disconnect — clear socket ID, mark captain unavailable
    socket.on('disconnect', async () => {
        await userModel.updateOne({ socketId: socket.id }, { $unset: { socketId: '' } });
        await captainModel.updateOne({ socketId: socket.id },
            { $unset: { socketId: '' }, $set: { isAvailable: false } }
        );
    });
});
```

2.5 GPS Tracking & Maps

The `LiveTracking` component uses `navigator.geolocation.watchPosition` with `enableHighAccuracy: true` to continuously stream the device GPS position. On each update, the TomTom Routing SDK recalculates the route, removes the stale GeoJSON polyline, and renders a fresh route layer on the canvas.

frontend/src/components/LiveTracking.jsx JSX

```jsx
const drawRoute = (origin, destination) => {
    ttServices.services.calculateRoute({
        key:       import.meta.env.VITE_TOMTOM_API_KEY,
        locations: [`${origin.lng},${origin.lat}`, `${destination.lng},${destination.lat}`],
    }).then((res) => {
        if (mapRef.current.getLayer('route')) {
            mapRef.current.removeLayer('route');
            mapRef.current.removeSource('route');
        }
        mapRef.current.addLayer({
            id: 'route', type: 'line',
            source: { type: 'geojson', data: res.toGeoJson() },
            paint:  { 'line-color': '#007aff', 'line-width': 6 }
        });
    });
};

useEffect(() => {
    const watchId = navigator.geolocation.watchPosition(
        ({ coords }) => {
            const pos = { lat: coords.latitude, lng: coords.longitude };
            mapRef.current.setCenter([pos.lng, pos.lat]);
            markerRef.current.setLngLat([pos.lng, pos.lat]);
            drawRoute(pos, destination);
        },
        (err) => console.error('GPS Error:', err),
        { enableHighAccuracy: true, timeout: 30000, maximumAge: 0 }
    );
    return () => navigator.geolocation.clearWatch(watchId);
}, [destination]);
```

2.6 Authentication (JWT + Bcrypt)

All protected routes are gated by middleware. Passwords are hashed using bcrypt (10 salt rounds). JWT tokens carry a 24-hour TTL. On logout, the token is inserted into a MongoDB blacklist — any subsequent request with a revoked token is rejected before JWT verification even runs.

backend/controllers/user.controller.js — Register & Logout JAVASCRIPT

```javascript
// Registration — hash password + issue JWT
module.exports.registerUser = async (req, res) => {
    const hashedPassword = await userModel.hashPassword(password);
    const user  = await userService.createUser({ ...fullname, email, password: hashedPassword });
    const token = user.generateAuthToken();  // JWT — 24h TTL
    await sendUserWelcomeEmail(email, fullname.firstname);
    res.status(201).json({ token, user });
};

// Logout — blacklist the token to prevent replay
module.exports.logoutUser = async (req, res) => {
    const token = req.cookies?.token || req.headers.authorization?.split(' ')[1];
    await blackListTokenModel.create({ token });  // Persist revoked token
    res.clearCookie('token');
    res.status(200).json({ message: 'Logged out successfully' });
};
```

© 2026 TravelX Team

2.7 Admin Analytics

The admin dashboard aggregates captain-level financial and operational statistics using MongoDB `$group` pipelines computed entirely on the database server. The frontend renders the returned data as a responsive bar chart using the Recharts library.

backend/services/ride.service.js — Earnings Aggregation JAVASCRIPT

```javascript
module.exports.getCaptainEarningsService = async (captainId) => {
    const result = await rideModel.aggregate([
        {
            $match: {
                captain: new mongoose.Types.ObjectId(captainId),
                status:  'completed'
            }
        },
        {
            $group: {
                _id:           null,
                totalFare:     { $sum: '$fare' },
                totaldistance: { $sum: '$distance' },
                totalRides:    { $sum: 1 }
            }
        }
    ]);
    return {
        totalFare: Math.round(result[0]?.totalFare || 0),
        totalRides: result[0]?.totalRides || 0
    };
};
```

frontend/src/pages/CaptainDashboard.jsx — Chart JSX

```jsx
<ResponsiveContainer width="100%" height="100%">
    <BarChart data={data.chartData}>
        <XAxis dataKey="name" fontSize={12} />
        <YAxis tickFormatter={(v) => Math.round(v)} />
        <Tooltip formatter={(v) => [Math.round(v), 'Earnings']} />
        <Bar dataKey="earnings" fill="#10B981" radius={[4,4,0,0]} />
    </BarChart>
</ResponsiveContainer>
```

© 2026 TravelX Team

03 Core Algorithms

ALGORITHM 3.1

Geospatial Driver Search

Uses MongoDB's native `$geoWithin` + `$centerSphere` to query captains within a radius. The radius in km is divided by 6,371 (Earth's mean radius) to convert to radians — the unit MongoDB requires for spherical geometry. No application-level distance loops are needed.

backend/services/maps.service.js JAVASCRIPT

```javascript
const getCaptainInTheRadius = async (lng, lat, radius) => {
    const captains = await captainModel.find({
        status:      'active',
        isAvailable: true,
        location: {
            $geoWithin: {
                $centerSphere: [[lng, lat], radius / 6371]  // km → radians
            }
        }
    });
    return captains;
};
```

ALGORITHM 3.2

Dynamic Fare Calculation

A three-variable pricing model: `fare = baseFare + (distance × perKmRate) + (duration × perMinuteRate)`. All three coefficients are defined per vehicle class. A single API call returns fares for all vehicle types simultaneously.

backend/services/ride.service.js JAVASCRIPT

```javascript
async function getFare(pickup, destination) {
    const distanceTime = await mapService.getDistanceTime(pickup, destination);
    const distance     = parseFloat(distanceTime.distance);  // km
    const duration     = parseFloat(distanceTime.duration);  // minutes

    const baseFare   = { auto: 30,  car: 50,  motorcycle: 20  };
    const perKmRate  = { auto: 10,  car: 15,  motorcycle: 8   };
    const perMinRate = { auto: 2,   car: 3,   motorcycle: 1.5 };

    return {
        auto:       baseFare.auto       + distance * perKmRate.auto       + duration * perMinRate.
auto,
        car:        baseFare.car        + distance * perKmRate.car        + duration * perMinRate.
car,
        motorcycle: baseFare.motorcycle + distance * perKmRate.motorcycle + duration * perMinRate.
motorcycle
    };
}
```

ALGORITHM 3.3

Cryptographic OTP Generation

Uses Node.js `crypto.randomInt` with a bounded range to guarantee exactly 6 digits with uniform distribution. The lower bound `10^(n-1)` prevents leading-zero collapse. The OTP is stored with `select: false` and is never returned in any API response.

backend/services/ride.service.js JAVASCRIPT

```javascript
function getOtp(num) {
    // crypto.randomInt — no modulo bias, uniform distribution
    return crypto.randomInt(
        Math.pow(10, num - 1),  // Lower: 100000
        Math.pow(10, num)        // Upper: 999999
    ).toString();
}

// Stored with select: false — never leaked in API responses
otp: getOtp(6)
```

ALGORITHM 3.4

Ride Lifecycle State Machine

Enforces strict state transitions: `pending → accepted → ongoing → completed`. `startRide` adds an OTP verification guard. `endRide` adds a captain identity lock — only the assigned captain can close the ride.

backend/services/ride.service.js JAVASCRIPT

```javascript
// accepted → ongoing  (OTP verification step)
module.exports.startRide = async ({ rideId, otp, captain }) => {
    const ride = await rideModel.findOne({ _id: rideId })
        .populate('user captain').select('+otp');  // Unlock hidden OTP field

    if (ride.status !== 'accepted') throw new Error('Ride not yet accepted.');
    if (ride.otp    !== otp)        throw new Error('Invalid OTP.');

    await rideModel.findOneAndUpdate({ _id: rideId }, { status: 'ongoing' });
    sendMessageToSocketId(ride.user.socketId, { event: 'ride-started', data: ride });
};

// ongoing → completed  (captain identity lock)
module.exports.endRide = async ({ rideId, captain }) => {
    const ride = await rideModel.findOne({ _id: rideId, captain: captain._id })
        .populate('user captain').select('+otp');

    if (ride.status !== 'ongoing') throw new Error('Ride is not ongoing.');
    await rideModel.findOneAndUpdate({ _id: rideId }, { status: 'completed' });
    sendMessageToSocketId(ride.user.socketId, { event: 'ride-ended', data: ride });
};
```

© 2026 TravelX Team

ALGORITHM 3.5

JWT Authentication with Token Blacklist

Three-layer auth pipeline: extract token → probe MongoDB blacklist → cryptographic JWT verification. All three must pass before the decoded identity is attached to the request context. Blacklisting on logout prevents replay attacks even within the token's remaining TTL.

backend/middlewares/auth.middleware.js JAVASCRIPT

```javascript
module.exports.authUser = async (req, res, next) => {
    // Layer 1: Extract token — cookie-first, header fallback
    const token = req.cookies.token || req.headers.authorization?.split(' ')[1];
    if (!token) return res.status(401).json({ message: 'Unauthenticated' });

    // Layer 2: Blacklist probe — reject revoked tokens immediately
    const isBlacklisted = await blacklistTokenModel.findOne({ token });
    if (isBlacklisted) return res.status(401).json({ message: 'Unauthorized' });

    try {
        // Layer 3: Cryptographic JWT verification
        const decoded = jwt.verify(token, process.env.JWT_SECRET);
        req.user = await userModel.findById(decoded._id);
        return next();
    } catch (err) {
        return res.status(401).json({ message: 'Unauthorized' });
    }
};
```

ALGORITHM 3.6

Bcrypt Password Hashing Pre-Save Guard

A Mongoose `pre('save')` hook uses a regex against the bcrypt format (`$2a/$2b/$2y`) to detect if a value is already hashed — preventing double-hashing corruption when a document is saved multiple times.

backend/models/user.model.js JAVASCRIPT

```javascript
userSchema.pre('save', async function (next) {
    if (!this.isModified('password')) return next();

    // Regex — detect existing bcrypt digest to skip re-hashing
    const isBcryptHash = /^\$2[aby]\$\d{2}\$.{53}$/.test(this.password);
    if (!isBcryptHash)
        this.password = await bcrypt.hash(this.password, 10);
    next();
});

userSchema.methods.generateAuthToken = function () {
    return jwt.sign({ _id: this._id }, process.env.JWT_SECRET, { expiresIn: '24h' });
};
```

© 2026 TravelX Team

04 Frontend Logic & UI Algorithms

4.1 Debounce-Controlled Location Search

A hand-rolled debounce suppresses rapid keystrokes and fires the TomTom autocomplete API only after 200 ms of inactivity. The function is memoized with `useCallback` to maintain a stable reference across React re-renders.

frontend/src/pages/Homee.jsx JSX

```jsx
const debounce = (func, delay) => {
    let timer;
    return (...args) => {
        clearTimeout(timer);
        timer = setTimeout(() => func(...args), delay);
    };
};

const fetchSuggestions = async (input) => {
    const token = localStorage.getItem('token');
    const res   = await fetch(
        `${import.meta.env.VITE_BASE_URL}/maps/get-suggestions?input=${encodeURIComponent(input)}
`,
        { headers: { Authorization: `Bearer ${token}` } }
    );
    const data = await res.json();
    setSuggestions(data?.predictions || []);
};

// Memoize — stable reference across re-renders
const debouncedFetchSuggestions = useCallback(debounce(fetchSuggestions, 200), []);
```

4.2 Captain GPS Polling Engine

A `setInterval`-driven loop fires every 10 seconds. Each tick requests the device GPS with `enableHighAccuracy: true` and emits the coordinates over the persistent WebSocket. The interval is cleaned up on component unmount to prevent memory leaks.

frontend/src/pages/CaptainHome.jsx JSX

```jsx
useEffect(() => {
    if (!captain?._id) return;
    socket.emit('join', { userId: captain._id, userType: 'captain' });

    const updateLocation = () => {
        navigator.geolocation.getCurrentPosition(
            ({ coords }) => {
                const loc = { lat: coords.latitude, lng: coords.longitude };
                setCaptainLocation(loc);
                // Emit live position to backend socket
                socket.emit('update-location-captain', { userId: captain._id, location: loc });
            },
            (err) => console.error('GPS error:', err.message),
            { enableHighAccuracy: true, timeout: 10000, maximumAge: 0 }
        );
    };

    updateLocation();                                          // Immediate first ping
    const interval = setInterval(updateLocation, 10000);       // Poll every 10 s
    return () => clearInterval(interval);                      // Cleanup on unmount
}, [captain, socket]);
```

4.3 Live Map Route Rendering (TomTom SDK)

On every `watchPosition` callback, any existing route layer is removed and the fresh GeoJSON polyline from the TomTom Routing API is re-rendered on the map canvas — giving the appearance of a continuously updating turn-by-turn route.

frontend/src/components/LiveTracking.jsx JSX

```jsx
const drawRoute = (origin, destination) => {
    ttServices.services.calculateRoute({
        key:       import.meta.env.VITE_TOMTOM_API_KEY,
        locations: [`${origin.lng},${origin.lat}`, `${destination.lng},${destination.lat}`],
    }).then((res) => {
        // Remove stale route before drawing the updated one
        if (mapRef.current.getLayer('route')) {
            mapRef.current.removeLayer('route');
            mapRef.current.removeSource('route');
        }
        mapRef.current.addLayer({
            id: 'route', type: 'line',
            source: { type: 'geojson', data: res.toGeoJson() },
            paint:  { 'line-color': '#007aff', 'line-width': 6 }
        });
    });
};
```

4.4 Live Distance Matrix Fetcher

On the captain's active ride screen, the remaining driving distance to the destination is fetched using the captain's live GPS fix. The backend proxies raw coordinates to TomTom Routing — bypassing geocoding. The response string (e.g. `"4.23 km"`) is regex-stripped and parsed to a float for display.

frontend/src/pages/CaptainRiding.jsx JSX

```jsx
const calculateDistance = async (currentLocation) => {
    const { lat: originLat, lng: originLng } = currentLocation;
    const destLat = Number(ride?.destLat);
    const destLng = Number(ride?.destLng);

    const response = await axios.get(`${import.meta.env.VITE_BASE_URL}/maps/getdistby`, {
        params:  { originLat, originLng, destLat, destLng },
        headers: { Authorization: `Bearer ${localStorage.getItem('token')}` }
    });

    const rawDistance = response.data?.distance;
    if (rawDistance && typeof rawDistance === 'string') {
        // Strip unit suffix (e.g. " km") — extract numeric portion only
        const num = parseFloat(rawDistance.replace(/[^\d.]/g, ''));
        if (!isNaN(num)) setDistance(num);
    }
};
```

05 Legal Note

⚖ COPYRIGHT NOTICE

This document contains selected core source code logic prepared exclusively for copyright registration purposes.

The code snippets presented herein represent the proprietary algorithms, business logic, and system architecture of the **TravelX – Real-Time Ride Booking Application**, developed by **Ujjwal Mutneja** along with the project team: **Vaibhav Tiwari**, **Sahibjot Singh**, and **Harshita Bansal**.

Full source code — including all frontend components, backend routes, database schemas, configuration files, and environment settings — is maintained privately and has not been published or distributed publicly.

Unauthorized reproduction, distribution, reverse engineering, or use of any part of this codebase without explicit written permission from the authors is strictly prohibited.

TravelX — Real-Time Ride Booking Web Application

© 2026 TravelX Team. All Rights Reserved.